Docker 部署语言 · Python 与 R

第 11 讲

目标

解释型语言, 直接放运行时镜像就跑

- 解释型 vs 编译型: **无显式编译 → 免多阶段**
- Python(重点): 官方镜像 + requirements + CMD
- R: r-base + 装包 + Rscript
- 层缓存顺序: 先装依赖、后放代码

一、解释型更"轻"在哪

> 没有编译, 就没有"编译产物"

编译型要产物, 解释型边解释边跑

```
编译型(C/Java): 源码 --编译⟶ 产物 --运行时⟶ 运行
                                    (编译器只在构建镜像)
解释型(Python/R): 源码 ———直接———⟶ 解释器 运行
```



> 所以解释型: 官方运行时镜像 + 源码 + 启动命令, 就够了

解释型部署通用三步

1. 选**官方运行时镜像**: `python` / `r-base`
2. **装依赖**: `requirements.txt` +pip / `install.packages()`
3. 写**启动命令**: `CMD` / `ENTRYPOINT`

> 三部曲, 每个解释型语言都一样

二、部署 Python (重点)

> 镜像自带解释器

一个零依赖脚本就三行

```
FROM python:3.12-slim
WORKDIR /app
COPY hello.py .
CMD ["python", "hello.py"]
```



```
docker build -t hello-py .
docker run --rm hello-py    # Hello from Python!
```



有依赖: requirements.txt 先装、代码后放

```dockerfile
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .                      # 先拷依赖清单
RUN pip install --no-cache-dir -r requirements.txt   # 再装
COPY app.py .                                # 最后放代码
EXPOSE 5000
CMD ["python", "app.py"]
```



顺序决定**层缓存**: 依赖没变, 就不会重跑 pip

Web 应用记住 host 0.0.0.0

```python
# app.py
from flask import Flask
app = Flask(__name__)
@app.route("/")
def home():
    return "Hello Flask in Docker!"
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```



> 容器外要能访问, 必须监听 0.0.0.0, 不是 127.0.0.1

ENTRYPOINT 固定入口, CMD 当默认参数

```
ENTRYPOINT ["python", "app.py"]   # 入口写死
CMD ["--port", "5000"]            # 默认参数, 运行时能改
```

- `CMD` : 启动命令, 可被 `docker run ... 命令` 覆盖
- `ENTRYPOINT` : 固定的可执行入口

三、部署 R

> 一个 r-base 镜像搞定

Rscript 跑脚本, 一个镜像出结果

```
FROM r-base:4.4
WORKDIR /work
COPY hello.R .
CMD ["Rscript", "hello.R"]
```

```
docker build -t hello-r .
docker run --rm hello-r    # [1] "Hello from R!"
```

> 一样是: 镜像 + 装包 + 启动命令

要装 R 包？装完再把结果交给 -v

```
FROM r-base:4.4
WORKDIR /work
COPY analyze.r .
RUN Rscript -e 'install.packages("dplyr", repos="https://cran.r-project.org")'
CMD ["Rscript", "analyze.r"]
```

```
mkdir -p out
docker run --rm -v "$PWD/out":/out hello-r
cat out/top.csv       # 结果落在宿主机
```

Python 与 R 同物一表

维度	Python	R
镜像	`python:3.12-slim`	`r-base`
依赖	pip + requirements	install.packages()
启动	`python app.py`	`Rscript xxx.R`
场景	Web/脚本	数据分析/报表
关键点	web 绑 0.0.0.0	结果 -v 导出

四、实战一

部署 Flask (作业)

> 有依赖的 Python Web

Flask 三步: 清单→安装→放码

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

```
docker build -t flask-app .
docker run -d -p 5000:5000 --name flk flask-app
curl http://localhost:5000    # Hello Flask from Docker
docker logs flk
```

五、实战二

R 分析出报表

> 容器算, 结果进宿主

算完把结果挂出来

```
# analyze.r
library(dplyr)
df ← data.frame(name=c("alice","bob"), score=c(90,85))
write.csv(df %>% filter(score > 80), "/out/top.csv", row.names=FALSE)
```



```
mkdir -p out
docker run --rm -v "$PWD/out":/out hello-r
cat out/top.csv      # alice,90  bob,85 —— 在宿主机
```



一张表带走全部要点

方面	要点
解释型	无显式编译 → **免多阶段**, 直接运行时镜像
三步	官方镜像 + 装依赖 + 启动命令
Python	requirements 先+code 后(**层缓存**) · 0.0.0.0
R	r-base + install.packages + Rscript
导出	R 结果 -v 到宿主
命令	CMD 可覆盖 · ENTRYPOINT 固定

解释型的"轻", 是直接把解释器装进镜像

下一步: 部署 Web 服务(Apache、Tomcat、Nginx) —— 中间件类镜像, 讲怎么选、怎么配、怎么挂网站

谢谢大家

Q & A